Embedded Software Test Part 1 - Concepts

Advanced Embedded Software Development

with **Dan Walkes**


University of Colorado **Boulder**

Learning objectives:

Introduction to Software Testing Concepts, Strategies, and Nomenclature

Understand role of Manual and Automated Testing of Embedded Devices.

Testing Strategies

- Manual Testing
  - Almost always required on Embedded Systems in at least some areas.
  - Typically start with manual tests for hardest portions to automate.
- Automated Testing
  - Obvious benefits for repeatability and speed

ISTQB Software Testing Principles

- Testing shows the presence of defects but not their absence
- It's not possible to exhaustively test
- Early testing saves time and money
- Defects are often clustered
- Tests need to be continually developed to continue to find defects "pesticide paradox"
- Testing is context dependent
  - Safety critical software isn't tested the same as a commercial mobile application
- Absence of Errors is a Fallacy
  - Just finding/fixing bugs doesn't mean the software achieves its objectives.

International Software Testing Qualifications Board
https://www.istqb.org/downloads/send/2-foundation-level-documents/281-istqb-ctfl-syllabus-2018-v3-1.html

Software Testing Definitions

- Test Case
  - Set of preconditions, inputs, actions, expected results, post conditions
- Test Execution
  - Activity that runs a test on a component or system producing results.
  - Linked to:
    - A specific version of the software
    - (in our case) a specific version of hardware
    - Possibly specific versions of external dependencies
- Test Suite
  - A set of test scripts or test procedures to be executed in a specific test run
    - Outcome: Test executions with same/similar software versions

https://glossary.istqb.org/en/search

Software Testing Levels

- Component Testing
  - Test individual software components in isolation
  - Often performed by the developer who wrote the code
- Integration testing
  - Test interaction between components or systems
- System Testing
  - Behavior and capabilities of an entire system or product
  - end-to-end tasks and functional behaviors
- Acceptance Testing
  - System Testing used to gauge readiness for deployment and use by the customer
  - Example: Alpha and Beta Testing at customer sites

International Software Testing Qualifications Board
https://www.istqb.org/downloads/send/2-foundation-level-documents/281-istqb-ctfl-syllabus-2018-v3-1.html

Manual Testing Strategies

- Record what you plan to do - Test Case
  - Use step-by-step detail.
  - Don’t require in depth software knowledge/tool access.
  - Include detail to allow someone other than a developer to run.
- Record what you did and when - Test Case Execution
  - Which software version
  - Dependencies/external devices/applications

Automated Testing Strategies

- Make the build automated
- Merge code from developers into a shared “mainline” branch often (Continuous Integration)
- Integrate test into the development environment
  - Run as many of these as automated as possible
  - Run all automated tests when new code is added to “mainline” to catch integration issues.

Embedded Test Part 2 - Automated Testing

Advanced Embedded Software Development

with **Dan Walkes**



University of Colorado **Boulder**

Learning objectives:

Understand components of an automated test framework.

Introduce Mocks

Introduce the Unity and GoogleTest automated test frameworks.

Automated Test Components

- Test Framework
  - Create executables on run platform
    - Often the development platform instead of the target platform
  - Group tests by features
  - Select tests to run
  - Report status
- Test Mocks

Automated Test Mocks

- Simulate the behavior of real objects in controlled ways
- Necessary with:
  - Non deterministic results (current time temperature)
  - States difficult to reproduce (device or network error)
  - Slow or requires setup (database interaction)
  - Does not exist or may change behavior
- Also useful for testing software components in isolation/isolating dependencies between components (Component Testing)

https://en.wikipedia.org/wiki/Mock_object

Unity Example

- Start with something interesting to test

```
/**
* Set @param result with the absolute time difference between @param time1 and @param time2, handling
* rollover.
*/
static inline void timespec_diff( struct timespec *result,
                        const struct timespec *time1, const struct timespec *time2)
{
        if( timespec_to_seconds(time2) >= timespec_to_seconds(time1) ) {
                timespec_diff_before_after(result,time1,time2);
        } else {
                timespec_diff_before_after(result,time2,time1);
        }
}
```

https://github.com/cu-ecen-5013/aesd-lectures/blob/master/lecture9/time_functions_shared.h

Unity Example

- Add some test functions with name test_ which include assertions in test/Test_time_functions_shared.c
- See https://github.com/ThrowTheSwitch/Unity/blob/master/docs/UnityAssertionsReference.md for complete list of assertions

```
+ void test_timespec_diff(void)
+ {
+     struct timespec time1;
+     struct timespec time2;
+     struct timespec result;
+     seconds_to_timespec(&time1,1.0);
+     seconds_to_timespec(&time2,1.1);
+     timespec_diff(&result,&time1,&time2);
+     TEST_ASSERT_EQUAL_FLOAT_MESSAGE(0.1,timespec_to_seconds(&result),"Expect timespec_diff of 1.1 -
1.0 to equal 0.1 seconds");
```

https://github.com/cu-ecen-5013/aesd-lectures/pull/1/files

Unity Example

- Generate a test runner using ruby and your test functions file
  - This finds the test_ prefix functions in your Test_XXXX.c file and runs them from an auto-generated main() function and executable

```
+ #!/bin/bash
+ # A script to auto generate test runners for unity tests
+ cd `dirname $0`
+ set -e
+ pushd ..
+ mkdir -p test/test_runners
+ # See https://github.com/ThrowTheSwitch/Unity/blob/master/docs/UnityHelperScriptsGuide.md
+ ruby Unity/auto/generate_test_runner.rb test/Test_time_functions_shared.c
test/test_runners/Test_time_functions_shared_Runner.c
```

https://github.com/cu-ecen-5013/aesd-lectures/pull/1/files
https://github.com/ThrowTheSwitch/Unity/blob/master/docs/UnityHelperScriptsGuide.md

Unity Example

- Modify the cmake configuration to:
  - Create a new project cmake-example-tests
  - Add Unity submodule (already includes CMakeLists.txt)
  - Add include directories for the root directory and Unity/src
  - Create a new executable unity-test which builds based on our c source files, test file and test runner
  - Link unity-tests with unity library

```
+ project(unity-test)
+ add_subdirectory(Unity)
+ include_directories(Unity/src .)
+ add_executable(unity-test test/Test_time_functions_shared.c
test/test_runners/Test_time_functions_shared_Runner.c)
+ target_link_libraries(unity-test unity)
```

https://github.com/cu-ecen-5013/aesd-lectures/pull/1/files

Unity Example

- Then build and run the unity-test executable using cmake

```
aesd@aesd-VirtualBox:~/aesd-lectures/lecture9$ mkdir build
aesd@aesd-VirtualBox:~/aesd-lectures/lecture9$ cd build
aesd@aesd-VirtualBox:~/aesd-lectures/lecture9/build$ cmake ..
-- The C compiler identification is GNU 7.5.0
-- The CXX compiler identification is GNU 7.5.0
-- Check for working C compiler: /usr/bin/cc
-- Check for working C compiler: /usr/bin/cc -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Detecting C compile features
-- Detecting C compile features - done
-- Check for working CXX compiler: /usr/bin/c++
-- Check for working CXX compiler: /usr/bin/c++ -- works
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- Configuring done
-- Generating done
-- Build files have been written to: /home/aesd/aesd-lectures/lecture9/build
aesd@aesd-VirtualBox:~/aesd-lectures/lecture9/build$ make
Scanning dependencies of target auto-generate-unity
~/aesd-lectures/lecture9 ~/aesd-lectures/lecture9/test
[  0%] Built target auto-generate-unity
Scanning dependencies of target unity
[ 20%] Building C object Unity/src/CMakeFiles/unity.dir/unity.c.o
[ 40%] Linking C static library libunity.a
[ 40%] Built target unity
Scanning dependencies of target unity-test
[ 60%] Building C object CMakeFiles/unity-test.dir/test/Test_time_functions_shared.c.o
[ 80%] Building C object CMakeFiles/unity-test.dir/test/test_runners/Test_time_functions_shared_Runner.c.o
[100%] Linking C executable unity-test
[100%] Built target unity-test
aesd@aesd-VirtualBox:~/aesd-lectures/lecture9/build$ ./unity-test
test/Test_time_functions_shared.c:7:test_timespec_add:PASS
test/Test_time_functions_shared.c:22:test_timespec_diff:PASS
test/Test_time_functions_shared.c:41:test_somefailure:FAIL: Expected 0.2 Was 0.1. this will fail

-----------------------
3 Tests 1 Failures 0 Ignored
FAIL
```

https://github.com/cu-ecen-5013/cmake-example/pull/1/files

CMock

- Creates mocks and stubs for C functions
- Creates “fake” versions of functions outside your module
- Uses ruby scripts to auto-generate c source corresponding to interfaces in a header file.
- For instance if your file1.c #includes “NeatStuff.h” which contains:

```
typedef struct neat_feature
{
  int someval;
  const char *somestring;
}NEAT_FEATURE_T;

int ParseStuff(char* Cmd);
void HandleNeatFeatures(NEAT_FEATURE_T NeatFeature);
```

- The auto generated sources include these function declarations and definitions:

```
typedef int (*CMOCK_ParseStuff_CALLBACK) (char *);
typedef void (*CMOCK_HandleNeatFeatures_CALLBACK) (NEAT_FEATURE_T *);

int ParseStuff(char* Cmd);
void ParseStuff_ExpectAndReturn(char* Cmd, int toReturn);
void ParseStuff_IgnoreAndReturn(int toReturn);
void ParseStuff_StubAndCallback(CMOCK_ParseStuff_CALLBACK Callback);

void HandleNeatFeatures(NEAT_FEATURE_T* NeatFeature);
void HandleNeatFeatures_Expect(NEAT_FEATURE_T* NeatFeature);
void HandleNeatFeatures_ExpectWithArrays(NEAT_FEATURE_T* NeatFeature, int NeatFeature_Depth);
void HandleNeatFeatures_Ignore(void);
void HandleNeatFeatures_StubAndCallback(CMOCK_HandleNeatFeatures_CALLBACK Callback);
```

http://www.throwtheswitch.org/cmock

CMock

- When your tested code in a C file that #includes “NeatStuff.h” calls ParseStuff(char *Cmd) during unit tests it gets redirected to the auto generated mock.
- Your test case can call:
  - ParseStuff_ExpectAndReturn to setup what it expects your code to pass in Cmd and simulate a return value in toReturn
  - ParseStuff_IgnoreAndReturn to ignore whatever was passed in Cmd and always return the value in toReturn
  - ParseStuff_StubAndCallback to point to a custom callback function which does it’s own special logic/assertions.

```
typedef int (*CMOCK_ParseStuff_CALLBACK) (char *);
int ParseStuff(char* Cmd);
void ParseStuff_ExpectAndReturn(char* Cmd, int toReturn);
void ParseStuff_IgnoreAndReturn(int toReturn);
void ParseStuff_StubAndCallback(CMOCK_ParseStuff_CALLBACK Callback);
```

http://www.throwtheswitch.org/cmock

Ceedling

- Ruby wrapper around CMock and Unity as well as CException (which simulates C++ exceptions in C).
- Automates steps associated with generating individual test files and mocks using ruby and the Rake framework
- See example project in https://github.com/cu-ecen-5013/ceedling-example/
  - Install ceedling
  - Create a new project using ceedling new <project>
  - Add your project files into the src directory.
  - Add your test files into the test directory
    - Use pattern test_filename.c
  - Run ceedling test:all

http://www.throwtheswitch.org/ceedling
https://github.com/ThrowTheSwitch/Ceedling/blob/master/docs/CeedlingPacket.md

GoogleTest

- Googletest: https://github.com/google/googletest
  - C++ centric test framework, which can also be used to test C files
  - Uses C++ streams for verbose exception handling
  - Includes a googlemock framework for C++ mocks
  - Used on OpenCV and ROS (Robot Operating System) codebases

```
ASSERT_EQ(x.size(), y.size()) << "Vectors x and y are of unequal length";

for (int i = 0; i < x.size(); ++i) {
  EXPECT_EQ(x[i], y[i]) << "Vectors x and y differ at index " << i;
}
```

https://github.com/google/googletest/blob/master/googletest/docs/primer.md
https://en.wikipedia.org/wiki/Google_Test

Embedded Test Part 3 - CI/CD

Advanced Embedded Software Development

with **Dan Walkes**



University of Colorado **Boulder**

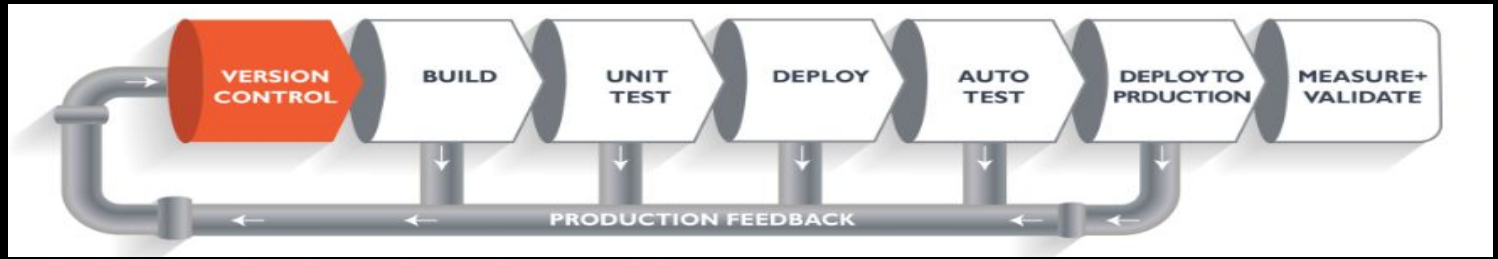
Learning objectives:

Introduce Continuous Integration and Continuous Deployment

Introduce CI frameworks Travis-ci and Gitlab CI

Continuous Integration / Continuous Deployment (CI/CD)

- When working with multiple developers, merge to a shared repository branch often, ideally daily.
- For each merge to shared branches, create a “CI Pipeline” to verify:
  - The build still succeeds
  - Automated tests still succeed
  - Code is able to be deployed to hardware
  - Automated tests on hardware succeed



https://codeship.com/continuous-integration-essentials
https://www.atlassian.com/continuous-delivery/continuous-integration
https://docs.travis-ci.com/user/for-beginners/

Jenkins



- Open source software for CI/CD intended to be self deployed
- Originally created in 2004
- Includes a Web UI for configuration/setup
- Uses text based “Jenkinsfile” to define your pipeline
- Can deploy to devices as a part of your pipeline
  - Good application for device update discussed in previous lectures!
- Other/newer options offer more tight integration with git revision control tools

https://jenkins.io/
https://en.wikipedia.org/wiki/Jenkins_(software)

YAML syntax

- YAML stands for “Yaml Ain’t Markup Language”
- Similar to JSON discussed in IoT lecture
  - JSON is designed for simplicity in generation and parsing “at the cost of reduced human readability”
  - YAML prioritizes human readability and support for serializing native data structures
  - Files are more readable than JSON but more complex to generate and parse
- Replacement for Extensible Markup Language (XML)
  - “YAML is the result of lessons learned from XML and other technologies”

JSON

```
{
  "person": {
    "firstname": "Tom",
    "lastname": "Smith",
    "year": 1982,
    "favorites": ["tennis", "golf"]
  }
}
```

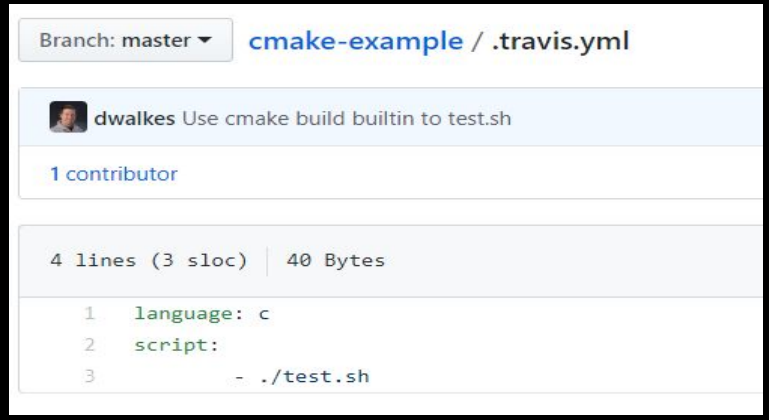
YAML

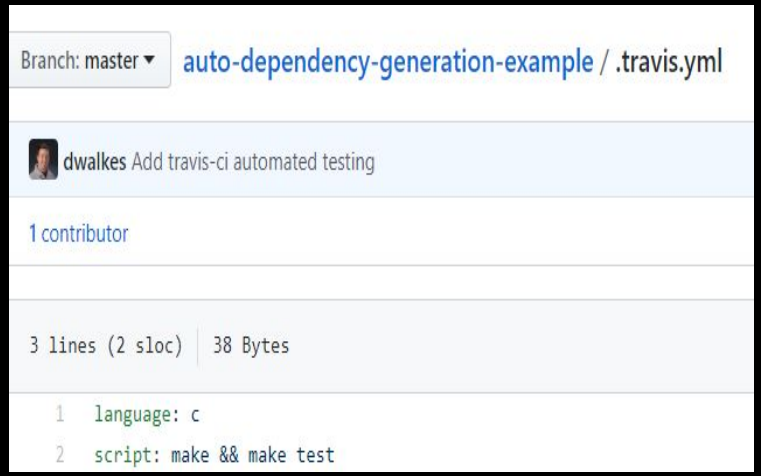
```
person:
  firstname: Tom
  lastname: Smith
  year: 1982
  favorites:
    - tennis
    - golf
```

https://yaml.org/spec/1.2/spec.html
http://sangsoonam.github.io/2017/03/13/yaml-vs-json.html

CI Frameworks - Travis-ci

- Hosted CI service which integrates well with GitHub
  - Free for open source repositories
  - Can't handle Yocto/buildroot builds
- Controlled by a simple Yaml file
  - language identifies program language
  - script identifies script to run when testing
    - Non-zero return from script indicates error





https://docs.travis-ci.com/user/tutorial/
https://docs.travis-ci.com/user/languages/c/
https://github.com/cu-ecen-5013/cmake-example/blob/master/.travis.yml
https://github.com/cu-ecen-5013/auto-dependency-generation-example/blob/master/.travis.yml

CI Frameworks - Gitlab CI

- Gitlab is an open-source alternative to github
  - Think of it as github in a self-hosted open source platform
  - Can also be used in hosted version at gitlab.com
- Has its own CI framework Gitlab-CI
  - Uses .yml syntax config files similar to Travis-ci
- Completely integrated with gitlab
  - automated builds start after each commit, report via same email/dashboards used for development.
- Possible to self-host gitlab ci “Runner” and cloud host gitlab server.
  - Gitlab runner could be on-premise, connected to physical hardware.

https://about.gitlab.com/
https://about.gitlab.com/stages-devops-lifecycle/continuous-integration/

Gitlab CI for AESD

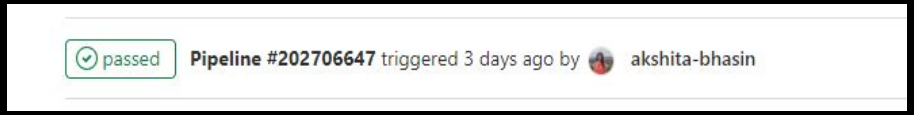
```yaml
image: "cuaesd/aesd-dind"

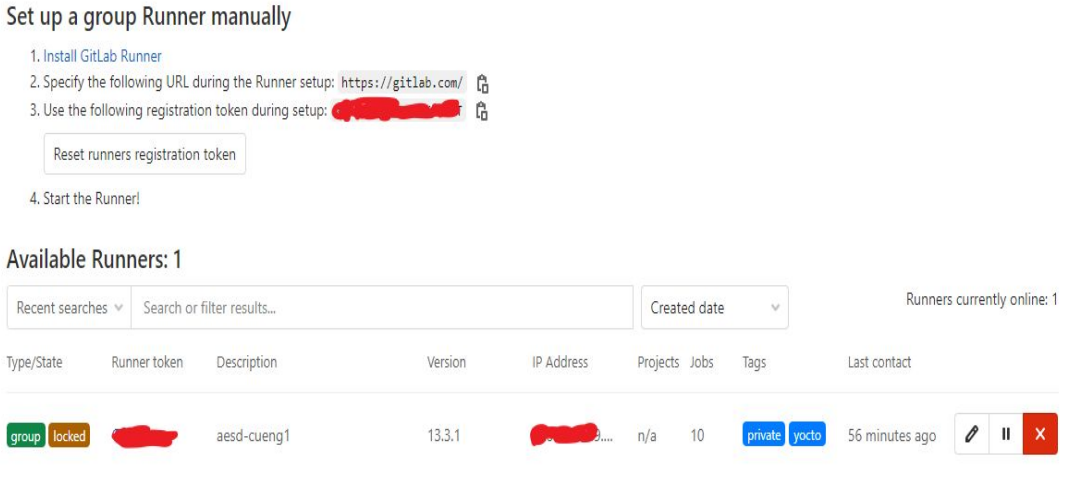
# See https://docs.gitlab.com/ee/ci/docker/using_docker_build.html#tls-enabled
services:
    - docker:19.03.12-dind

variables:
    GIT_SUBMODULE_STRATEGY: recursive
    # Specify to Docker where to create the certificates, Docker will
    # create them automatically on boot, and will create
    # `/certs/client` that will be shared between the service and job
    # container, thanks to volume mount from config.toml
    DOCKER_TLS_CERTDIR: "/certs"

test:
    stage: test
    script:
        - cd ${CI_PROJECT_DIR}
        - ./assignment-autotest/docker/docker-run-test.sh -n

    artifacts:
        paths:
            - ${CI_PROJECT_DIR}/test.sh.log
        when: always
```





https://github.com/cu-ecen-5013/yocto-assignments-base/blob/master/.gitlab-ci.yml

Embedded Test Part 4 - TDD and BDD

Advanced Embedded Software Development

with **Dan Walkes**



University of Colorado **Boulder**

Learning objectives:

Introduce Test Driven Development

Introduce Behavior Driven Development

Test Driven Development (TDD)

- Requirements written as test cases first in the test process.
- Test cases fail (since software isn't written yet)
- Write software
  - Test cases pass
- Pitfall:
  - Usually the developer writes tests for requirements
  - This assumes the developer understands/interprets the requirements properly
  - If not, both the test and the software could be implemented wrong, causing false success.

https://en.wikipedia.org/wiki/Test-driven_development

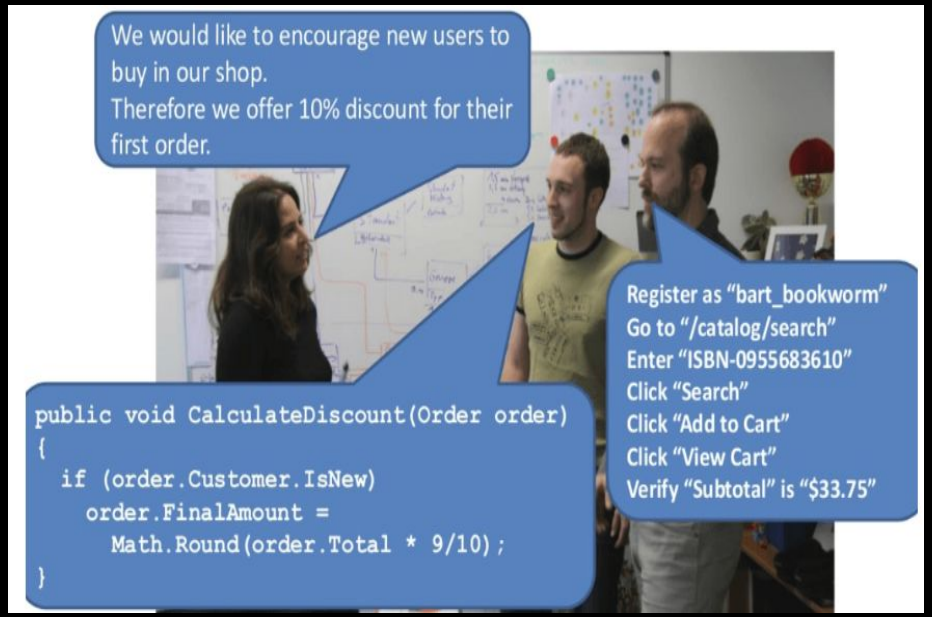
Behavior Driven Development (BDD)

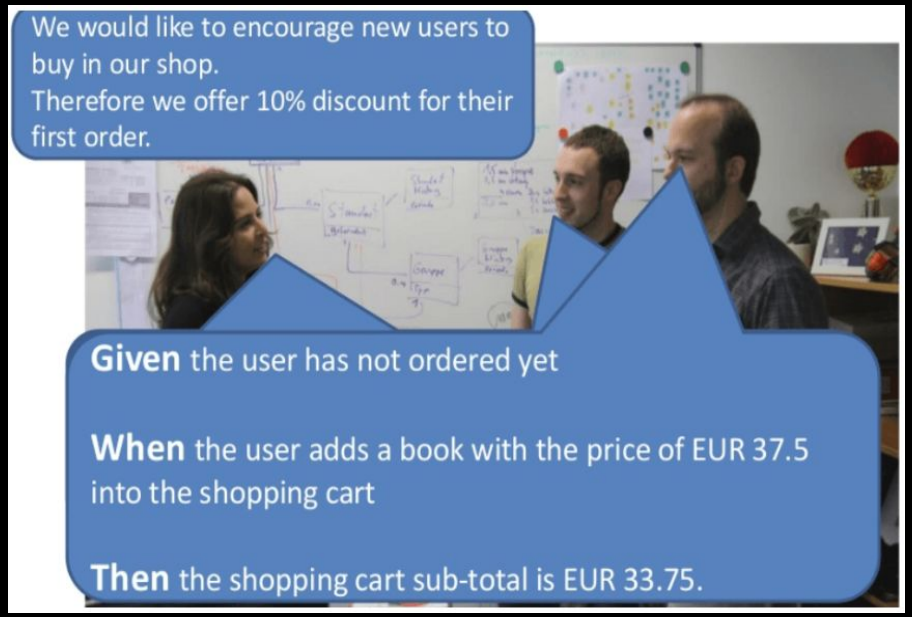
- Test cases are written in a natural language non-programmers can read, described as “scenarios”
- Scenarios describe the purpose and benefit of the code instead of technical details.
- Uses “Gherkin language” syntax

https://behave.readthedocs.io/en/latest/tutorial.html#python-step-implementations

Behavior Driven Development (BDD)

- Gherkin Language Comparison





https://www.guru99.com/gherkin-test-cucumber.html

Behavior Driven Development (BDD)

- behave: Python BDD framework
- Cucumber: Ruby BDD framework
- Example below is a behave implementation

```
Scenario: Search for an account
   Given I search for a valid account
    Then I will see the account details
```

```
@given('I search for a valid account')
def step_impl(context):
    context.browser.get('http://localhost:8000/index')
    form = get_element(context.browser, tag='form')
    get_element(form, name="msisdn").send_keys('61415551234')
    form.submit()

@then('I will see the account details')
def step_impl(context):
    elements = find_elements(context.browser, id='no-account')
    eq_(elements, [], 'account not found')
    h = get_element(context.browser, id='account-head')
    ok_(h.text.startswith("Account 61415551234"),
        'Heading %r has wrong text' % h.text)
```

https://behave.readthedocs.io/en/latest/tutorial.html#python-step-implementations